

ComfyLAB: Guia de Início Rápido e Visão Geral Completa

Comfortable Lab Automation Blocks — Manual do Usuário, Visão Geral do Software e Referência Prática

1. Introdução e Filosofia

1.1 O que é o ComfyLAB?

O **ComfyLAB** (**Com**fortable **L**ab **A**utomation **B**locks) é uma plataforma de programação visual de código aberto, projetada especificamente para automação de experimentos científicos, teste e medição, aquisição de dados (DAQ) e visualização em tempo real.

Em laboratórios de pesquisa modernos, sistemas de medição automatizados frequentemente exigem comunicação com diversos equipamentos (osciloscópios, fontes de alimentação, multímetros, geradores de funções, sensores customizados) por meio de múltiplos protocolos (VISA, SCPI, Serial RS-232/RS-485, Ethernet, bibliotecas nativas C/C++ DLLs). Escrever código de automação tradicional baseado em scripts geralmente resulta em códigos repetitivos (boilerplate) para gerenciamento de loops, segurança entre threads, tratamento de exceções, parsing de dados e renderização de interfaces gráficas.

O ComfyLAB resolve esse desafio fornecendo uma interface gráfica limpa e modular, onde os usuários arrastam **Blocos** funcionais para uma tela digital, os conectam através de **Fios** visuais e organizam resultados em um **Dashboard** interativo imediato.

1.2 Arquitetura Dupla: Interface Web UI + Motor de Execução Python

O ComfyLAB opera em um modelo cliente-servidor desacoplado, separando a interação do usuário da execução e comunicação de hardware:

Camada	Função Principal	Recursos Principais	Tecnologias
Interface Web (Front-End)	Interação e Visualização	- Canvas infinito com conexão drag-and-drop de fios - Cockpit dedicado de Dashboard do Operador (D) - Gráficos 2D/3D em tempo real e lousa interativa	React, XY Flow, Plotly
API de Comunicação	Transporte de Dados	- Streaming bidirecional de telemetria via WebSocket - Endpoints REST para controle de execução e status	WebSockets, FastAPI
Servidor (Back-End)	Execução e Hardware	- Executor de máquina de estados híbrida push/pull - Gerenciador de locks VISA e portas seriais - Servidor de Instrumentos Virtuais embutidos (SCPI) - Matemática NumPy/SciPy e scripts políglotas	Python, PyVISA, NumPy, SciPy

- Canvas Front-End & Dashboard:** Executa em navegadores web modernos (ou em builds de aplicativo desktop de arquivo único). Proporciona manipulação gráfica fluida, ferramentas de auto-layout, zoom/pan interativo em gráficos, anotações virtuais em lousa e um **Painel de Dashboard do Operador** que exibe cards fixados, controles e monitores

em uma interface limpa de sala de controle.

- **Servidor Back-End:** Alimentado por Python e FastAPI. Gerencia bloqueios de hardware em tempo real, consultas VISA/SCPI, cálculos matriciais pesados, execução multi-thread, persistência de arquivos, ambientes virtuais Python e um processo de servidor em segundo plano para **Instrumentos Virtuais**.
-

1.3 Segurança e Proteção Integradas

- **Proteção de Instrumentos e Rotinas de Segurança (Teardown):** Equipamentos físicos de laboratório (lasers, fontes de alta tensão, fontes de sinal RF) exigem gerenciamento de segurança rigoroso. Se uma execução de medição encontrar um erro, for interrompida manualmente ou sofrer perda de conexão, o ComfyLAB executa automaticamente rotinas de desligamento seguro (shutdown hooks) para desativar saídas ativas e retornar os instrumentos a estados seguros.
- **Verificação de Segurança de Blueprints:** Ao abrir blueprints de fontes externas, o ComfyLAB realiza verificações de segurança para evitar a execução não autorizada de scripts, garantindo controle total e confirmação rápida ao usuário antes de rodar códigos customizados.
- **Acesso Remoto Seguro:** O ComfyLAB suporta operação remota segura via rede com autenticação por token, permitindo que pesquisadores monitorem e controlem com segurança experimentos rodando em computadores do laboratório a partir de estações de trabalho remotas.
- **Verificador Integrado de Versões e Atualizações:** Gerenciador de atualizações embutido na janela *Sobre* (About), que verifica automaticamente novas versões no PyPI / GitHub, exibe notas de versão e permite atualização simplificada.

2. Capacidades e Flexibilidade: O que Pode (ou Não) Ser Feito

O ComfyLAB foi desenvolvido do zero para oferecer extrema flexibilidade, sem restringir usuários avançados a modelos de interface rígidos.

2.1 O que Pode Ser Feito com o ComfyLAB

Área de Recurso	Capacidades e Destaques
Controle de Hardware	Suporte completo a VISA (GPIB, USB-TMC, TCP/IP, Serial RS-232/485) e SCPI. Drivers integrados para osciloscópios, DMMs, fontes e espectrômetros.
Instrumentos Virtuais	Osciloscópio (<code>virt_osc</code>), gerador de funções (<code>virt_siggen</code>) e circuito RC simulados. Testes e ensino offline sem hardware físico.
Dashboard do Operador	Visualização dedicada em cockpit (atalho <code>D</code>). Fixe qualquer bloco ou gráfico em Cards de Controle e Monitoramento organizados.
Tempo e Progresso	Bloco Countdown Wait (com botão pular), barras de progresso (0–100%), relógio ETR e loops inteligentes com saídas de <code>%</code> e ETR.
Gráficos Científicos	2D e 3D em tempo real: XY (linear/log), Dual-Y, Barras, Box plot, Histograma, Superfície/Dispersão 3D, Polar, Cascata e Heatmaps.
Bibliotecas Nativas	Invocação direta de C/C++ (<code>.dll</code> no Windows, <code>.so</code> no Linux/macOS) via Editor de Assinaturas interativo, sem wrappers Python.
Scripts Políglotas	Blocos de script customizados em 9 linguagens: Python, Rust, JS, TS, Julia, R, Lua, Octave e Wolfram. Acesso a <code>COMFYLAB_WORKSPACE</code> .
Ecossistema Python	Conexão transparente com ambientes virtuais (<code>.venv</code>), permitindo qualquer pacote PyPI (<code>scipy</code> , <code>pandas</code> , <code>torch</code> , <code>opencv</code>).
Publicação e Pacotes	Publique blocos customizados diretamente na paleta lateral ou empacote fluxos completos em arquivos redistribuíveis <code>.cfy</code> .
Processamento de Sinais	Matemática matricial NumPy, espectro FFT, ajuste de curvas (Gaussiana, exponencial, polinomial), filtros digitais e detecção de picos.
Sub-Canvases	Agrupamento de sub-grafos complexos em blocos de Cluster modulares e reutilizáveis com pinos dinâmicos de E/S.
Camada de Lousa	Desenhos livres, notas adesivas e caixas geométricas de agrupamento diretamente sobre o canvas (atalho <code>4</code>).

2.2 Limitações Realistas: O que Não Pode (ou Não Deve) Ser Feito

Embora o ComfyLAB suporte quase qualquer tarefa de medição científica, automação e processamento de dados, é importante compreender seus limites de aplicação:

- **Determinismo de Tempo Real Estrito em Microsegundos:** O ComfyLAB executa sobre sistemas operacionais convencionais (Windows/Linux/macOS) através de back-ends em Python/FastAPI. Ele é projetado para loops de automação na escala de milissegundos a sub-segundos, amostragem de sensores e controle de instrumentos. Ele **não** se destina a lógicas de hardware bare-metal em FPGA ou loops de controle determinísticos de microsegundos em tempo real (ex.: comutação PWM de drivers de motor a 100 kHz). Para essas tarefas, placas FPGA ou microcontroladores dedicados com RTOS devem gerenciar a temporização de baixo nível, enquanto o ComfyLAB se comunica com eles via serial/USB para controle de alto nível, armazenamento em buffer e plotagem.

Resumo: Fora o determinismo de hardware em microsegundos via FPGA, **praticamente qualquer fluxo de medição científica, automação de laboratório, registro de dados, análise de sinais ou plotagem pode ser construído no ComfyLAB.**

3. Comparação com Paradigmas Alternativos

Compreender como o ComfyLAB se compara às ferramentas tradicionais de laboratório destaca por que ele é uma escolha vantajosa tanto para o ensino quanto para a pesquisa.

Recurso / Aspecto	Python Puro	NI LabVIEW	MATLAB / Simulink	ComfyLAB
Licença e Custo	Gratuito e Código Aberto	Comercial de alto custo	Comercial de alto custo	Gratuito e Código Aberto (GPLv3)
Paradigma	Script textual	Gráfico proprietário G	Diagrama de blocos / script	Diagrama Web + Código Poliglota
Curva de Aprendizado	Alta (sintaxe, APIs)	Íngreme (regras da ling. G)	Moderada (toolboxes)	Baixa (Drag-and-drop visual)
Interface e Gráficos	Manual (PyQt, Tkinter)	Painel Frontal Integrado	Janelas de Figura	Gráficos 2D/3D & Dashboard (D)
Simulação Offline	Mocks manuais	Mocks básicos de software	Motor físico do Simulink	Instrumentos Virtuais SCPI Embutidos
Arquivos e Git	Texto plano .py (Git-friendly)	Binário .vi (Conflitos no Git)	.m / Binário .slx	Blueprints JSON Limpos (Git-friendly)
Código Customizado	Nativo em Python	Nós complexos C/Python	Funções S / MATLAB	Scripts em 9 Linguagens e DLL/SO
Arquitetura	Script monolítico	Aplicação desktop pesada	Ambiente desktop pesado	Interface Web + Servidor FastAPI
Locks de Hardware	Implementação manual	Integrado ao DAQmx	Instrument Toolbox	Gerenciador de Lock VISA Assíncrono

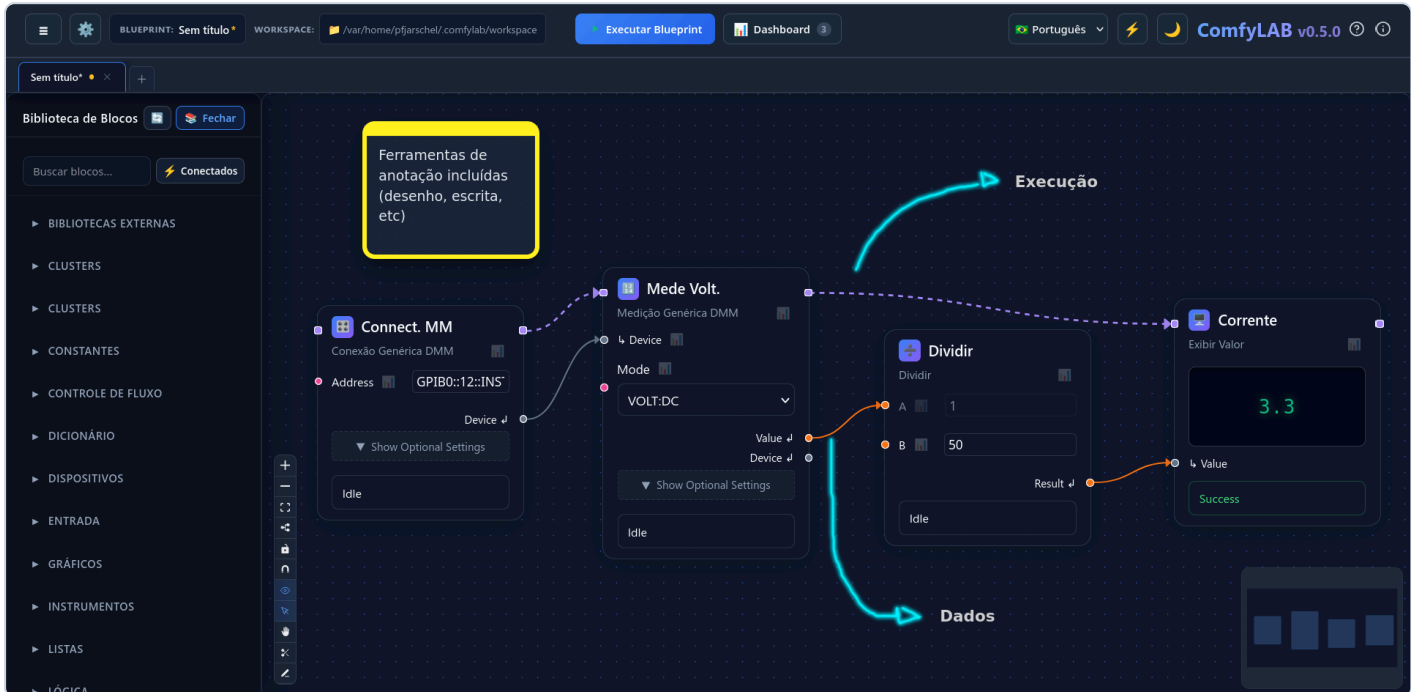
Principais Conclusões:

- Vs. Python Puro:** O Python é extremamente poderoso, mas criar interfaces de usuário, gráficos interativos em tempo real, loops de estado de hardware e fluxos visuais exige centenas de linhas de código repetitivo. O ComfyLAB mantém todo o poder analítico do Python, fornecendo uma interface visual imediata e cockpit de operação.
- Vs. LabVIEW:** O LabVIEW é amplamente utilizado, mas sofre com custos elevados de licença comercial, arquivos binários proprietários que dificultam o controle de versão e bloqueio do fornecedor (lock-in). O ComfyLAB oferece uma alternativa moderna e open-source com blueprints em JSON, instrumentos virtuais embutidos e integração nativa com navegadores web.
- Vs. MATLAB:** O MATLAB se destaca em cálculos matriciais, mas possui custos de licença elevados. O ComfyLAB utiliza back-ends gratuitos e de código aberto baseados em NumPy/SciPy.

4. Layout da Interface e Conceitos Fundamentais

4.1 Visão Geral do Layout da Interface

A interface do usuário do ComfyLAB é composta por três zonas funcionais principais cercadas por controles superiores:



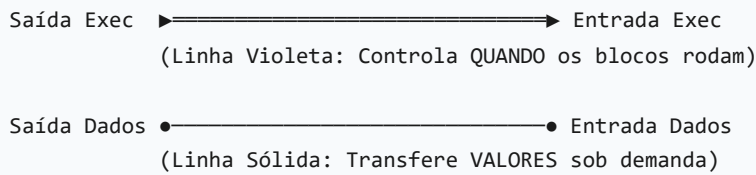
Tela geral do ComfyLAB, com a interface em português

Descrição Detalhada da Interface:

- **Barra de Ferramentas Superior:** Controles para executar (▶ / Ctrl+R), pausar (⏸) e parar (⏹ / Ctrl+Shift+R) blueprints, abrir o **Dashboard do Operador** (📊 / D), criar/salvar arquivos de workspace (Ctrl+S), alternar temas de cores e configurar opções globais do VISA e diagnósticos.
- **Barra Lateral (Esquerda):** Paleta pesquisável contendo todos os blocos nativos disponíveis, blocos customizados publicados, blocos de instrumentos virtuais e nós de script categorizados por domínio.
- **Canvas Principal (Centro):** Área de trabalho visual infinita com zoom e pan onde você arrasta, posiciona e conecta os blocos funcionais.
- **Inspetor de Blocos (Direita):** Painel de detalhes contextual que se abre ao selecionar um bloco. Exibe valores de pinos ao vivo, parâmetros do bloco, documentação resumida, notas e logs de console de execução.
- **Barra de Abas e Breadcrumbs:** Navegação entre múltiplos arquivos de blueprint abertos e detalhamento (drill down) em clusters de sub-canvas.

4.2 Pinos e Conexões: Fluxo de Execução vs. Valores de Dados

As conexões entre blocos no ComfyLAB são estritamente divididas em duas categorias distintas:



1. Pinos de Execução (Fluxo):

- Representados por **conectores em forma de seta** e interligados por **linhas violetas** (■).
- Ditam o **sequenciamento temporal e fluxo de controle** (quando as etapas executam, loops repetem ou ramificações são acionadas).

2. Pinos de Dados (Valores):

- Representados por **pinos circulares coloridos** e interligados por **linhas coloridas sólidas**.
- Transportam **valores** (números, strings de texto, vetores NumPy, dicionários, handles de hardware).
- São avaliados de forma preguiçosa (lazy evaluation) sob demanda quando um bloco de execução solicita dados de entrada.

Referência de Cores dos Pinos de Dados:

- Número (Laranja)**: Valores numéricos escalares inteiros ou de ponto flutuante.
- Booleano (Verde)**: Flags lógicas `True` (Verdadeiro) ou `False` (Falso).
- String (Rosa)**: Textos, caracteres formatados ou comandos SCPI.
- Array (Amarelo)**: Vetores ou matrizes numéricas NumPy 1D/2D de alto desempenho.
- Lista (Ciano)**: Coleções ordenadas de elementos.
- Dicionário (Marrom)**: Mapeamentos de parâmetros chave-valor.

4.3 Lógica de Execução e Cadeias Independentes

- Gatilho de Execução**: Clicar em **Run Blueprint** (▶ ou `Ctrl+R`) inicia a execução nos blocos de entrada (blocos sem conexões de execução de entrada) e segue os fios de execução fluxo abaixo.
- Cadeias de Execução Independentes**: Se o canvas contiver grupos de blocos separados e não conectados entre si, o ComfyLAB os trata automaticamente como cadeias de execução paralelas, permitindo que loops de aquisição independentes rodem concorrentemente.
- Blocos de Controle de Fluxo e Temporização**:
 - For Loop**: Executa um sub-loop por uma quantidade fixa de iterações, emitindo o índice atual, porcentagem de conclusão (%) e estimativa de tempo restante (ETR).
 - For Each Loop**: Itera sobre cada item em uma lista ou array, fornecendo saídas de item, índice, porcentagem e tempo restante (ETR).
 - While Loop**: Executa continuamente um sub-loop enquanto a condição Booleana do pino permanecer `True`.
 - If / Else Branch**: Direciona o fluxo de execução para uma saída `True` ou `False` com base em uma condição.
 - Delay / Sleep**: Pausa o fluxo de execução por uma duração especificada (ms ou segundos).
 - Countdown Wait**: Pausa a execução com relógio digital regressivo ao vivo, barra de progresso em esgotamento e botão de clique para **Pular Espera**.
 - Barra de Progresso**: Widget visual dedicado para acompanhamento de progresso de 0% a 100%.

- **Relógio ETR:** Mostrador digital estilo sala de controle que formata o tempo restante estimado (HH:MM:SS ou MM:SS).
- **Medidor de Tempo (Measure Time):** Mede com precisão o tempo decorrido de relógio (wall-clock) entre pulsos de execução.

4.4 Recursos Avançados

- **Painel de Dashboard e Cards Fixados** ( / ): O Dashboard proporciona um cockpit de operação limpo e sem distrações para conduzir experimentos, apresentar resultados ou operar bancadas experimentais sem a complexidade visual do diagrama de fios.
 - **Fixando Blocos no Dashboard:** Qualquer bloco ou pino pode ser fixado no Dashboard. Basta clicar com o botão direito no bloco e escolher **Pin to Dashboard**, clicar no ícone de alfinete no cabeçalho do Inspetor de Blocos ou usar o botão correspondente nos widgets suportados.
 - **Organização Automática:** Controles e entradas numéricas são reunidos em **Cards de Controle** interativos, enquanto gráficos, medidores e temporizadores formam **Cards de Monitoramento**.
 - **Personalização de Layout:** Reordene cards verticalmente, arraste os cantos dos gráficos para redimensioná-los ou clique no ícone de salto para navegar instantaneamente até o bloco no canvas.
 - **Visão Maximizada:** Maximize o painel do dashboard para ocupar todo o workspace em um painel de bancada em tela cheia.
- **Blocos Persistentes** (): Por padrão, os estados dos blocos são reiniciados a cada execução. Marcar um bloco como **Persistente** preserva seu estado interno e handles de hardware (ex.: conexões VISA ativas, portas seriais abertas ou médias móveis) entre múltiplas execuções do blueprint.
- **Desativação de Blocos** (): Clicar com o botão direito em um bloco para desativá-lo o deixa opaco no canvas. Blocos desativados são ignorados durante a execução, repassando dados de forma limpa sem executar código—ideal para testar fluxos sem hardware físico.
- **Clusters (Sub-canvases)** (): Agrupamento de arranjos complexos de blocos em um único bloco customizado. Dar um duplo clique em um bloco de cluster abre um sub-canvas isolado, com breadcrumbs na barra superior para navegação.
- **Modo Lousa / Whiteboard** (): Clique no ícone de ferramenta ou pressione o atalho  4 para alternar o modo de desenho, permitindo adicionar notas adesivas, caixas de texto e formas geométricas diretamente sobre o canvas.

5. Resumo das Categorias de Blocos Nativos

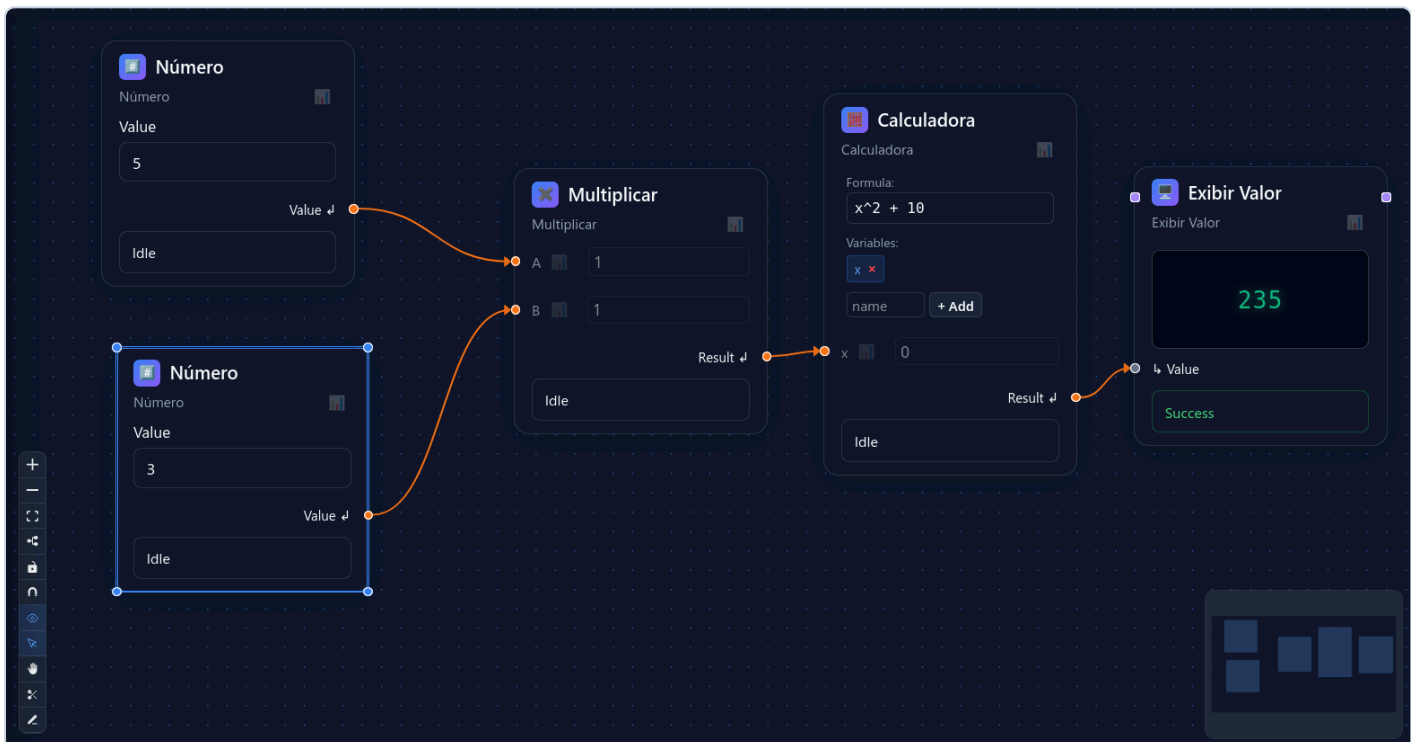
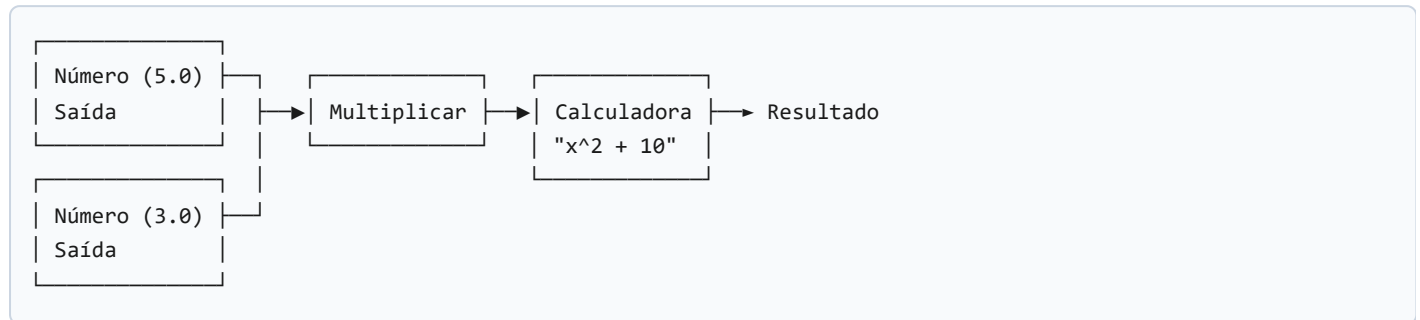
Categoria	Função Principal e Blocos Chave
 Controle de Fluxo e Tempo	Orquestração de loops e temporização: <code>Laço For</code> (<code>For Loop</code>), <code>Laço For Each</code> , <code>Laço While</code> , <code>Se/Senão</code> (<code>If/Else</code>), <code>Aguardar</code> (<code>Sleep</code>), <code>Espera com Contagem</code> (<code>Countdown Wait</code>), <code>Medir Tempo</code> , <code>Temporizador</code> .
 Matemática e Lógica	Aritmética, lógica booleana e equações: <code>Adicionar</code> , <code>Subtrair</code> , <code>Multiplicar</code> , <code>Dividir</code> , <code>Potência</code> , <code>Calculadora</code> (<code>Calculator</code>), <code>Trigonometria</code> , comparações (<code>></code> , <code><</code> , <code>==</code>), portas lógicas (<code>AND</code> , <code>OR</code> , <code>NOT</code>).
 Dados e Strings	Manipulação de texto e dicionários: <code>Concatenar</code> , <code>Formatar String</code> , <code>Dividir String</code> , <code>Substituir String</code> , <code>Criar Dicionário</code> , <code>Obter Valor</code> , <code>Definir Chave/Valor</code> .
 Arrays e Sinais	Processamento de matrizes numéricas e sinais: <code>Criar NDArray</code> , <code>Linspace</code> , <code>Fatiar NDArray</code> , <code>Espectro FFT</code> (<code>FFT Spectrum</code>), <code>Filtro de Sinal</code> , <code>Ajuste de Curva</code> (<code>Curve Fit</code>).
 Entrada/Saída de Arquivos	Persistência e exportação de dados: <code>Salvar CSV</code> (<code>Save CSV</code>), <code>Carregar CSV</code> , <code>Salvar JSON</code> , <code>Carregar JSON</code> , <code>Salvar Parquet</code> , <code>Carregar Parquet</code> , <code>Imagem para Array</code> .
 VISA e Hardware Físico	Comunicação com instrumentos: <code>Dispositivo VISA</code> (<code>VISA Device</code>), <code>Consulta VISA</code> (<code>VISA Query</code>), <code>Leitura VISA</code> (<code>VISA Read</code>), <code>Escrita VISA</code> (<code>VISA Write</code>), <code>Gerenciador de Recursos VISA</code> .
 Instrumentos Virtuais	Simulação sem hardware: <code>Conexão VirtOsc</code> , <code>Adquirir VirtOsc</code> , <code>Conexão VirtSigGen</code> , <code>Configurar Onda VirtSigGen</code> , <code>Circuito RC Simulado</code> .
 Drivers de Fabricantes	Drivers integrados: Osciloscópios (Tektronix, Keysight), Multímetros (HP/Agilent 34401A), Fontes DC, Espectrômetros Horiba, Digitalizadores CAEN, Motores Thorlabs.
 Exibição e Dashboard	Visualização de dados e monitoramento: <code>Gráfico XY</code> (<code>XY Plot</code>), <code>Gráfico de Tempo</code> , <code>Gráfico de Eixo Y Duplo</code> , <code>Gráfico Box / Violin</code> , <code>Gráfico de Barras</code> , <code>Histograma</code> , <code>Gráfico 3D</code> , <code>Gráfico Polar</code> , <code>Espectrograma Cascata</code> , <code>Mapa de Calor</code> , <code>Barra de Progresso</code> , <code>Relógio ETR</code> .
 Scripts e Nativo	Extensões customizadas: <code>Script Python</code> (<code>Python Script</code>), <code>Python Externo</code> , <code>Script JavaScript</code> , <code>Script TypeScript</code> , <code>Script Julia</code> , <code>Script Rust</code> , <code>Script Lua</code> , <code>Script Octave</code> , <code>Script R</code> , <code>Script Wolfram</code> , <code>Carregar DLL/SO</code> .

6. Tutoriais Práticos de Iniciação

Siga estes exemplos práticos passo a passo para dominar rapidamente os fluxos fundamentais do ComfyLAB.

Tutorial 1: Pipeline Matemático e Inspeção ao Vivo

Objetivo: Criar dois números, multiplicá-los, avaliar uma expressão matemática usando o bloco **Calculadora** (`Calculator`) e inspecionar o resultado no Inspetor de Blocos.



Exemplo 1

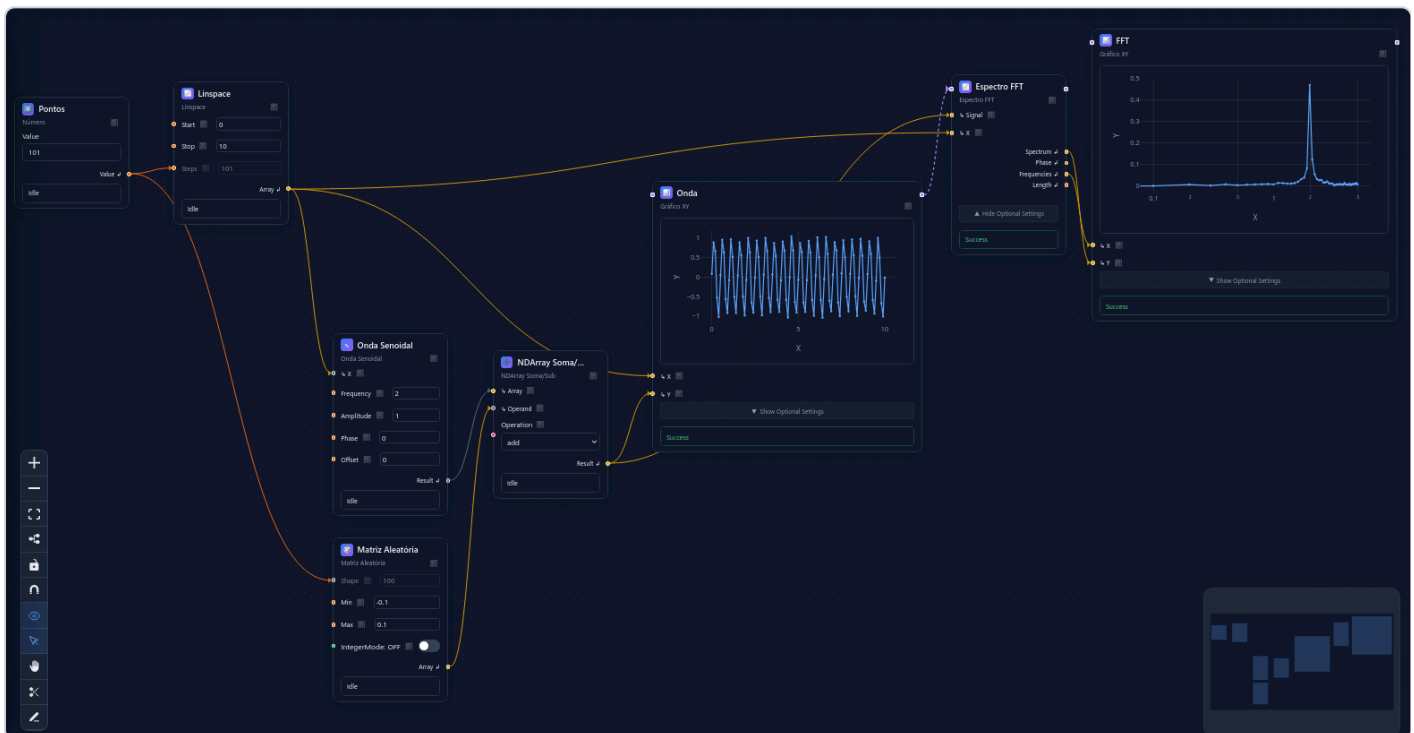
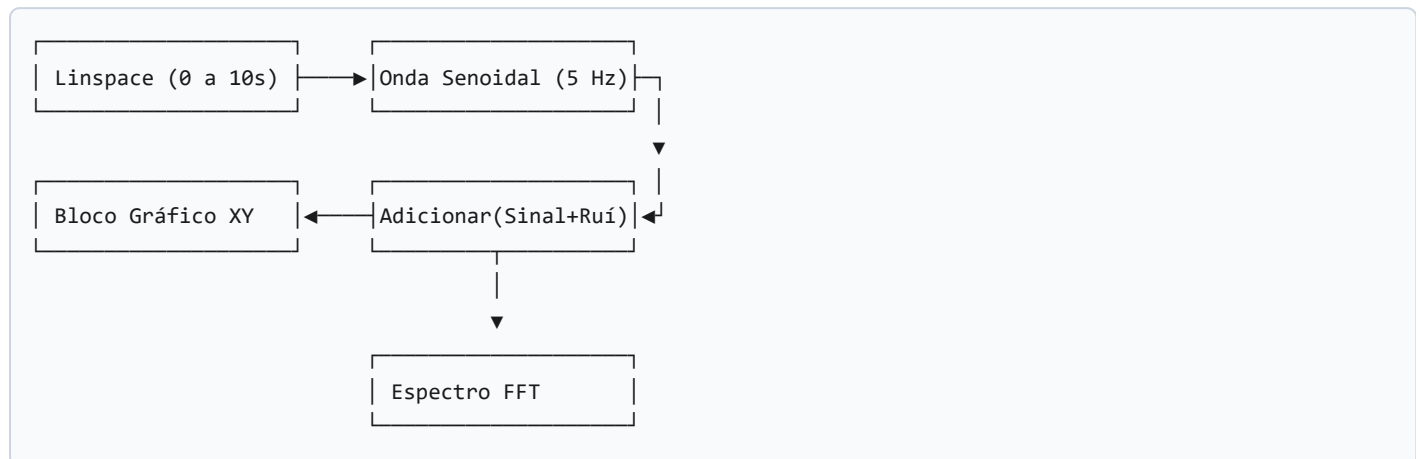
Instruções Passo a Passo:

1. Abra o ComfyLAB e crie um novo workspace/blueprint.
2. Na **Barra Lateral** à esquerda, busque por **Número** (`Number` , em *Constantes*) e arraste **dois** blocos Número para o canvas.
3. Defina o valor do primeiro bloco Número para `5.0` e o do segundo para `3.0` .
4. Arraste um bloco **Multiplicar** (`Multiply` , em *Matemática / Básica*) para o canvas.
5. Conecte o pino de saída de dados do Número 1 à Entrada A do bloco Multiplicar, e o Número 2 à Entrada B.
6. Arraste um bloco **Calculadora** (`Calculator` , em *Matemática / Básica*) para o canvas. No Inspetor de Blocos, defina a expressão para `x^2 + 10` e adicione a variável `x` .
7. Conecte o pino de saída do bloco Multiplicar (`15.0`) ao pino de entrada `x` do bloco Calculadora.
8. Clique em **Executar Blueprint** (▶ ou `Ctrl+R`) na barra de ferramentas superior.
9. Clique no bloco **Calculadora** para abrir o **Inspetor de Blocos** no painel da direita.

10. **Verificação:** No Inspetor de Blocos, verifique o valor no pino de saída **Resultado** (`Result`). Ele deve avaliar para $(15)^2 + 10 = 235.0$.

Tutorial 2: Geração de Sinal Sintético e Plotagem ao Vivo

Objetivo: Gerar um array de onda senoidal ruidosa, calcular seu espectro FFT e exibi-lo em um bloco **Gráfico XY** (`XY Plot`).



Exemplo 2

Instruções Passo a Passo:

- Busque por **Linspace** (em **Arrays Numéricos**) na Barra Lateral. Defina **Start** = 0, **Stop** = 10, **Points** = 1000 para criar o vetor de tempo.
- Busque por **Onda Senoidal** (**Sine Wave** , em **Matemática / Funções**). Conecte o array de saída do Linspace à entrada **X** do bloco Onda Senoidal. Defina **Frequência** = 5.0 Hz.
- Adicione um bloco **Matriz Aleatória** (**Random Array** , em **Matemática / Aleatório**, defina **Forma** = 1000, **Mín** = -0.2, **Máx** = 0.2) e um bloco **Adicionar** (**Add**) para somar o ruído à senoide.
- Arraste um bloco **Gráfico XY** (**XY Plot** , em **Gráficos**) para o canvas.
- Conecte o array de tempo do Linspace ao pino **X** do Gráfico XY e o array ruidoso ao pino **Y**. Conecte uma linha de execução ao pino **Plotar** (**Plot**).
- (Opcional) Conecte o sinal a um bloco **Espectro FFT** (**FFT Spectrum** , em **Matemática / Processamento de Sinais**) para visualizar o espectro de frequências.

7. Clique em **Executar Blueprint** ( ou `Ctrl+R`).

8. **Verificação:** O widget do Gráfico XY exibirá imediatamente a onda senoidal ruidosa em tempo real. Use a roda do mouse sobre o gráfico para aproximar/afastar (zoom) e arraste para navegar pelos eixos!

Tutorial 3: Varredura Automatizada de Parâmetros e Registro em CSV

Objetivo: Usar um **Laço For** (For Loop) para varrer valores de tensão, calcular a corrente e registrar os dados em um arquivo CSV com o bloco **Salvar CSV** (Save CSV).



Instruções Passo a Passo:

1. Arraste um bloco **Laço For** (For Loop , em *Controle de Fluxo*) para o canvas.
2. Defina `Iterações = 50` . Observe que o bloco disponibiliza saídas de `%` (porcentagem) e `ETR` (tempo restante estimado)!
3. Dentro da execução do loop, multiplique o pino `Índice` (Index) por `0.1` usando um bloco **Multiplicar** (Multiply) para gerar uma varredura de `0.0 V` a `5.0 V` .
4. Divida a tensão por uma resistência constante (`R = 100 Ohms`) com um bloco **Dividir** (Divide) para calcular a corrente `I` .
5. Arraste um bloco **Salvar CSV** (Save CSV , em *Entrada / Saída*) para o canvas. Defina `Caminho do Arquivo` (FilePath) para `sweep_results.csv` .
6. Conecte os dados ao pino `Dados` (Data) do Salvar CSV. Conecte a linha de execução `Corpo` (Body) do loop ao pino `Escrever` (Write) do Salvar CSV.
7. Conecte o pino de saída de execução `Saída` (Out) do Salvar CSV de volta à entrada `Passo` (Step) do loop para avançar a iteração.
8. Clique em **Executar Blueprint** (▶ ou `Ctrl+R`).
9. **Verificação:** Observe o loop executando passo a passo. Após a conclusão, abra a pasta do seu workspace ativo—o arquivo `sweep_results.csv` estará salvo com as 50 linhas registradas!

Tutorial 4: Criação e Publicação de um Bloco de Script Python Customizado

Objetivo: Criar um bloco **Script Python** (`Python Script`) customizado que filtra valores abaixo de um limiar, testá-lo e publicá-lo na paleta da barra lateral esquerda.

```
Bloco Script Python
Entradas: data_in (Array), threshold (Número)
Saídas:  filtered_data (Array), count (Número)

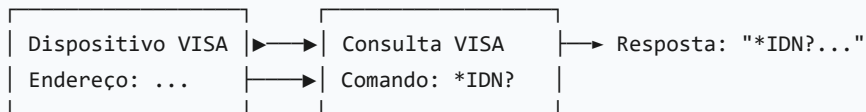
arr = np.array(inputs['data_in'])
result = arr[arr > inputs['threshold']]
outputs['filtered_data'] = result.tolist()
outputs['count'] = len(result)
```

Instruções Passo a Passo:

1. Arraste um bloco **Script Python** (`Python Script` , da categoria *Scripts*) para o canvas.
2. No **Inspetor de Blocos**, clique em **Editar Pinos** (`Edit Pins`):
 - Adicionar Pino de Entrada: `data_in` (Tipo: Array)
 - Adicionar Pino de Entrada: `threshold` (Tipo: Number)
 - Adicionar Pino de Saída: `filtered_data` (Tipo: Array)
 - Adicionar Pino de Saída: `count_passed` (Tipo: Number)
3. Dê um duplo clique no bloco Script Python para abrir o editor de código Monaco integrado.
4. Digite o código Python mostrado acima. (Observe que `os.environ["COMFYLAB_WORKSPACE"]` está disponível caso seu script precise acessar arquivos do projeto!).
5. Conecte um array de amostra em `data_in` e defina `threshold = 2.5` .
6. Clique em **Executar Blueprint** () e inspecione os pinos `filtered_data` e `count_passed` no Inspetor de Blocos.
7. Após verificar, clique em **Publicar Bloco** (`Publish Block`) no Inspetor de Blocos. Dê o nome de `Filtro Limiar` , escolha um ícone e atribua à categoria `Arrays` .
8. **Verificação:** Observe a sua **Barra Lateral** à esquerda. Seu novo bloco `Filtro Limiar` agora está permanentemente disponível para ser arrastado para qualquer canvas no seu workspace!

Tutorial 5: Interface de Hardware VISA (Consulta SCPI de Identificação)

Objetivo: Conectar a um instrumento via VISA usando o bloco **Dispositivo VISA** (`VISA Device`), enviar uma consulta com **Consulta VISA** (`VISA Query`) e exibir a resposta de identificação do modelo.

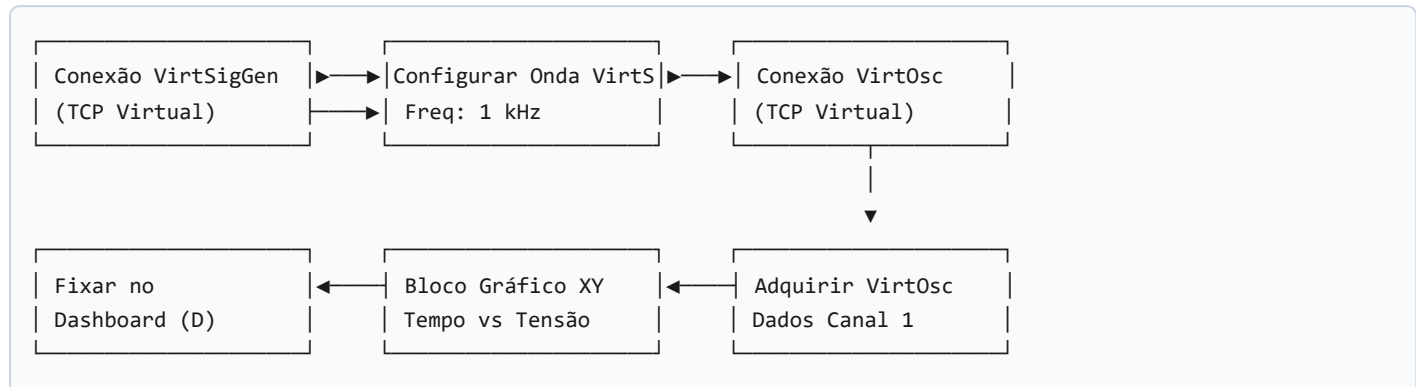


Instruções Passo a Passo:

1. Arraste um bloco **Dispositivo VISA** (`VISA Device` , em *Instrumentos / VISA*) para o canvas. Defina o parâmetro **Endereço** (`Address`) para o endereço do seu instrumento (ex.: `TCPIP0::192.168.1.100::INSTR` , `GPIB0::14::INSTR` , `COM3` ou `VIRT::OSC`).
2. Arraste um bloco **Consulta VISA** (`VISA Query` , em *Instrumentos / VISA*) para o canvas.
3. Conecte o pino de execução **Saída** (`Out`) do Dispositivo VISA à entrada **Entrada** (`In`) da Consulta VISA.
4. Conecte o pino de handle **Dispositivo** (`Device` , pino ciano) do Dispositivo VISA à entrada **Dispositivo** (`Device`) da Consulta VISA.
5. Defina o parâmetro **Comando** (`Command`) na Consulta VISA para `*IDN?` .
6. Clique em **Executar Blueprint** ( ou `Ctrl+R`). (O ComfyLAB gerencia automaticamente locks de acesso e fechamento seguro).
7. **Verificação:** Clique no bloco **Consulta VISA** e observe o Inspetor de Blocos. O pino de saída **Resposta** (`Response`) exibirá a identificação do fabricante (ex.: `HEWLETT-PACKARD,34401A,0,11-5-2` ou `KEYSIGHT TECHNOLOGIES,DSOX2002A,...`).
8. **Dica:** Você também pode utilizar o bloco **Gerenciador de Recursos VISA** (`VISA Resource Manager`) para descobrir instrumentos conectados automaticamente!

Tutorial 6: Automação Offline com Instrumentos Virtuais Embutidos

Objetivo: Conectar aos instrumentos virtuais do ComfyLAB, configurar um gerador de sinais simulado, capturar a forma de onda com o osciloscópio virtual e monitorar os resultados no Dashboard sem precisar de hardware físico.



Instruções Passo a Passo:

1. Na Barra Lateral, navegue até a categoria **Dispositivos > Virtual**.
2. Arraste um bloco **Conexão VirtSigGen** (VirtSigGen Connect) e um bloco **Configurar Onda VirtSigGen** (VirtSigGen Config Wave). Conecte-os para gerar uma onda senoidal de 1,0 kHz com 2,0 Vpp.
3. Arraste um bloco **Conexão VirtOsc** (VirtOsc Connect) e um bloco **Adquirir VirtOsc** (VirtOsc Acquire). Conecte as linhas de execução e os handles dos dispositivos.
4. Arraste um bloco **Gráfico XY** (XY Plot). Conecte a saída de tempo do osciloscópio em **X** e o traço de tensão em **Y**.
5. Clique com o botão direito no bloco **Gráfico XY** e selecione **Fixar no Dashboard** (Pin to Dashboard).
6. Pressione **D** no teclado (ou clique no botão **Dashboard** na barra superior) para abrir o cockpit de operação.
7. Clique em **Executar Blueprint** (▶ ou **Ctrl+R**).
8. **Verificação:** O ComfyLAB inicializará automaticamente o servidor de simulação de instrumentos virtuais em segundo plano. O Dashboard exibirá imediatamente a forma de onda capturada em tempo real. Você também pode carregar o exemplo pronto **Bode Diagram Virtual** em **Menu Arquivo > Carregar Exemplo > Bode_Diagram_Virtual.json** para ver uma varredura completa de resposta em frequência com traçado de diagramas de Bode!

7. Guia Rápido de Atalhos e Referência de Teclado

Atalhos de Teclado

Ação	Atalho	Descrição
Alternar Dashboard	D	Abrir ou fechar o cockpit do Dashboard do Operador
Executar / Pausar / Retomar	Ctrl + R	Iniciar a execução do blueprint, pausar ou retomar
Parar Execução	Ctrl + Shift + R	Interromper a execução imediatamente e acionar rotinas de segurança
Duplicar Bloco(s)	Ctrl + D	Clonar blocos selecionados preservando suas configurações
Ferramenta Seleção	1	Modo padrão: selecionar, mover blocos e conectar pinos
Ferramenta Pan	2	Mover a visualização do canvas sem arrastar blocos
Ferramenta Cortar Fio	3	Passar a lâmina sobre os fios para excluí-los rapidamente
Ferramenta Lousa	4	Alternar camada de desenho para notas, formas e anotações
Mover Canvas (Pan)	Espaço + Arrastar	Segurar barra de espaço e arrastar o fundo
Zoom no Canvas	Roda do Mouse	Rolar para aproximar ou afastar a visualização
Salvar Blueprint	Ctrl + S	Salvar o plano atual no JSON do workspace (Ctrl + Shift + S para Salvar Como)
Abrir Blueprint	Ctrl + O	Abrir caixa de diálogo para carregar blueprint
Desfazer / Refazer	Ctrl + Z / Ctrl + Y	Desfazer ou refazer ações recentes no canvas
Copiar / Colar	Ctrl + C / Ctrl + V	Copiar e colar blocos selecionados
Excluir	Delete / Backspace	Remover blocos ou fios selecionados
Navegar em Cluster	Duplo Clique	Entrar no sub-canvas de um Bloco de Cluster